I have a process that's archiving all bluesky posts (https://github.com/g-clef/bluesky_consumer), and I would like to set up a query layer on the resulting parquet files, to allow an LLM agent to query posts, run analyses against them, etc. I could theoretically allow the agents to simply open the files, but that seems inefficient, especially since I want the agent to be able to do long term, longitudinal studies of account behavior.

What are some good ways I can expose the data in a set of parquet files created by the bluesky consume to an LLM agent running in the same kuberentes cluster to allow the agent to do multi-month queries?

Let me first look at the repo to understand the data structure and how the parquet files are organized.

Fetched: GitHub - g-clef/bluesky_consumer: a consumer of the bluesky firehose for post analysis >

Good, now I have a clear picture. Let me look at the storage worker code to understand the exact parquet schema.

Searched the web >

Now I have a solid understanding of your setup. Let me lay out the options.

Based on the repo, here's what you're working with: parquet files partitioned as `s3://bluesky-data/year=YYYY/month=MM/day=DD/hour=HH/*.parquet` in MinIO, with columns like `collection_timestamp`, `event_timestamp`, `did`, `event_type`, `collection`, `action`, `record_text`, `reply_parent`, `reply_root`, `embed_type`, and `raw_event`. You want an LLM agent in the same K8s cluster to run analytical queries spanning months of data.

Here are the main approaches, roughly ordered from lightest to heaviest:

1. DuckDB over S3 — the simplest good option

Deploy a small service (or sidecar) that wraps DuckDB's S3-native parquet reading. DuckDB handles partition pruning on your `year=/month=/day=` structure natively, so a query like "all posts from DID X between January and March" only reads the relevant files. The agent sends SQL strings, the service returns results.

Why this is appealing for your case: zero data duplication, no ETL pipeline, no additional stateful infrastructure. DuckDB handles multi-file glob queries like `read_parquet('s3://bluesky-data/year=2025/**/*.parquet')` efficiently, and the Hive-style partitioning means it prunes aggressively. You could expose it as a simple REST API or, even better, as an MCP server so the agent can call it as a tool.

The main downside is that DuckDB is single-node. For the volume of the full Bluesky firehose (thousands of events/second), after a few months you'll have a *lot* of data. DuckDB can handle surprisingly large datasets (hundreds of GB) on a reasonably provisioned pod, but there's a ceiling.

2. DuckDB with a materialized layer

Same as above, but you add a periodic job that pre-aggregates common patterns into summary parquet files: per-DID daily activity counts, posting frequency distributions, reply graph snapshots, etc. The agent queries the summaries for longitudinal analysis and drops into the raw data only when it needs specific posts. This dramatically reduces the scan volume for the multi-month queries you're describing and keeps DuckDB viable even at very large scale.

3. ClickHouse

If you anticipate the data volume growing past what a single DuckDB node handles comfortably (or you want concurrent queries from multiple agents), ClickHouse is a natural step up. It's a columnar OLAP database that can ingest parquet directly from S3, handles the kind of aggregation queries your agent would run (group by DID over time ranges, count patterns, etc.) extremely well, and runs nicely in Kubernetes via the

ClickHouse Operator. You'd set up a periodic ingest job that loads new parquet files into ClickHouse tables, or use ClickHouse's S3 table engine to query them in place.

The trade-off is operational complexity — you're now running a database cluster. But ClickHouse is pretty lightweight compared to Spark or Trino for this kind of workload.

4. Trino / Spark with a Hive Metastore

This is the "enterprise" option. You deploy Trino (or Spark Thrift Server) with a Hive metastore that catalogs your partitioned parquet data in MinIO. This gives you a full SQL interface with excellent partition pruning, and scales horizontally. Trino has the advantage of being a query engine only (no data copying), while Spark gives you more flexibility for complex transformations.

This is probably overkill for your use case unless you're also serving other analytical workloads. The operational burden of metastore + coordinator + workers is significant for a homelab.

5. Apache Iceberg (or Delta Lake) table format

Rather than raw partitioned parquet, you convert your data into an Iceberg table on MinIO. This gives you proper table semantics (schema evolution, time travel, compaction) while still storing everything as parquet under the hood. You can then query it with DuckDB, Trino, Spark, or ClickHouse — any of them understand Iceberg. The storage workers could write directly to Iceberg instead of raw parquet, or you run a periodic compaction job.

This is worth considering if you find yourself wanting to update or delete data (e.g., removing posts from accounts that get suspended), but it adds a layer of abstraction that may not be necessary yet.

My recommendation for your setup:

Start with **option 2** — DuckDB behind a thin API service, plus a daily aggregation job. Here's why:

You're already on a homelab K8s cluster with MinIO, so adding another stateful system (ClickHouse, Trino) increases your ops burden significantly. DuckDB needs one pod, zero external dependencies, and directly queries your existing parquet files in MinIO. The materialized summary tables handle the "longitudinal study" requirement without scanning terabytes of raw data each time the agent asks about posting patterns over six months.

For the agent interface specifically, I'd expose the DuckDB service as an **MCP server** with tools like `query_posts(sql)`, `get_account_summary(did, start_date, end_date)`, and `search_text(keywords, date_range)`. This gives the agent